

Malware

Shuai Wang



香港科技大學

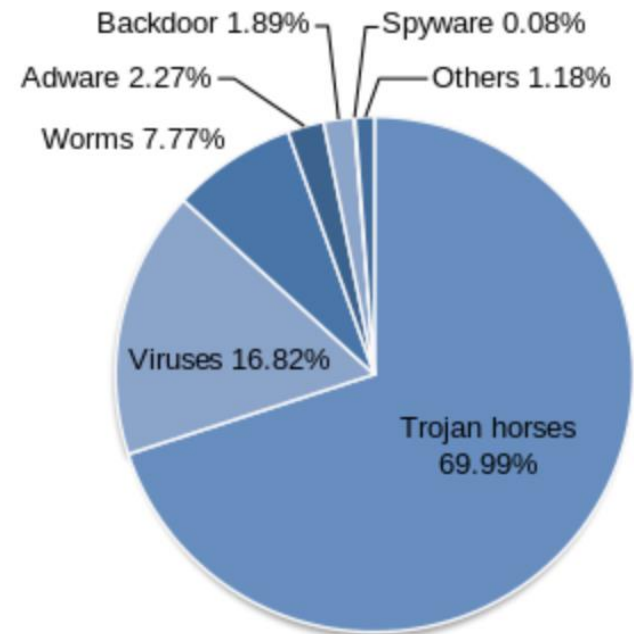
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

Malicious Software

- Adversaries aim to **get code running on your computer** that performs tasks of their choosing
 - This code is often called malware
 - What can they do?
 - Steal personal information
 - Delete files
 - Click fraud
 - Steal software serial numbers
 - Use your computer as relay

Malicious Software

- Adversaries aim to **get code running on your computer** that performs tasks of their choosing
 - Types of malware (no **standard** definition)
 - Worm/Virus — active or passive propagation
 - Trojan horse — unexpected functionality
 - Trapdoor/backdoor — unauthorized access
 - ...



Morris Worms

- A worm is a **self-propagating** program through the **Internet**.
- What it tried to do
 - Exploits some vulnerability on a target host ...
 - embeds itself into the host ...
 - Searches for other vulnerable hosts ...
 - Do it again until you take down the whole network!

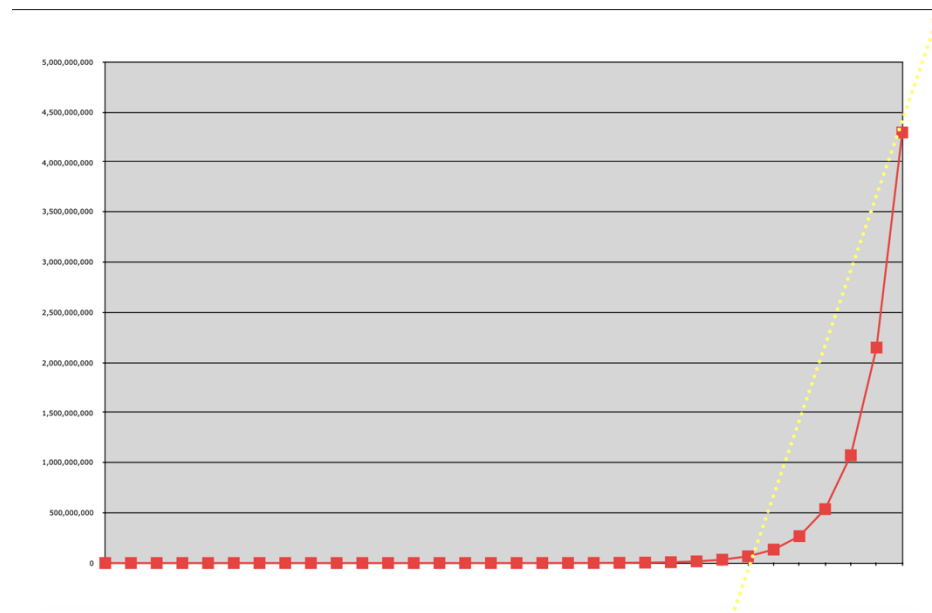
Morris Worm 1988

- Robert Morris, a 23 doctoral student from Cornell
 - Wrote a small (99 line) program
 - Launched on November 3rd, 1988
 - Simply disabled the Internet
- Obtained access to machines by...
 - Exploit bugs in fingerd and sendmail (buffer overflow)
 - Guess user password (a list of common password)
- Flaws in fingerd and sendmail were well-known
 - but not widely patched



Worms

- What makes worms so dangerous is that infection grows at an exponential rate



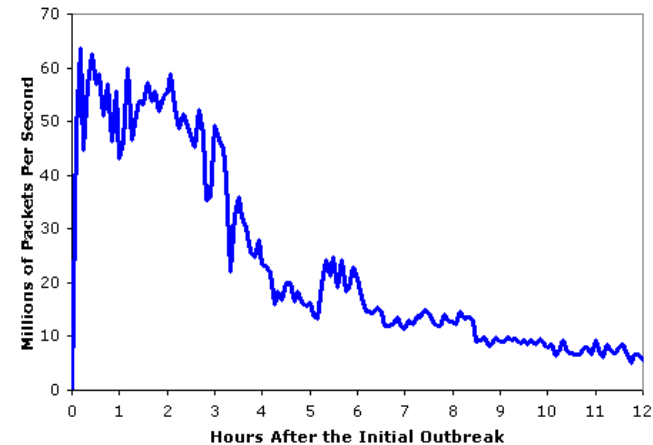
Morris Worm

- Shock to the Internet community of 1988
 - Internet of 1988 *much* different than today
- Internet designed to survive *nuclear war*
 - Yet, brought down by one graduate student!
 - At the time, Morris' father worked at NSA...
- Could have been much worse
- Result? More security awareness

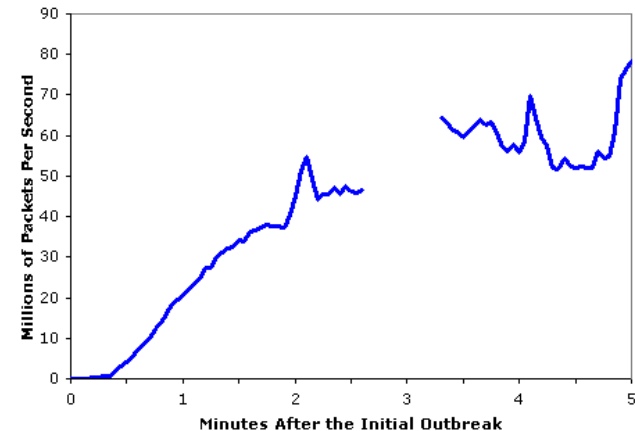
SQL Slammer -- 2003

- Infected **75,000 systems in 10 minutes!**
 - A tiny worm within one network package
 - One 376-byte UDP package
 - Firewall often let one package thru
 - A bug in database software
 - Although that patch has been released 6 months ago.
- At its peak, infections doubled every 8.5 seconds
- Spread “too fast” so it “burned out” available bandwidth

Aggregate Scans/Second in the 12 Hours After the Initial Outbreak



Aggregate Scans/Second in the first 5 minutes based on Incoming Connections To the WAIL Tarpit

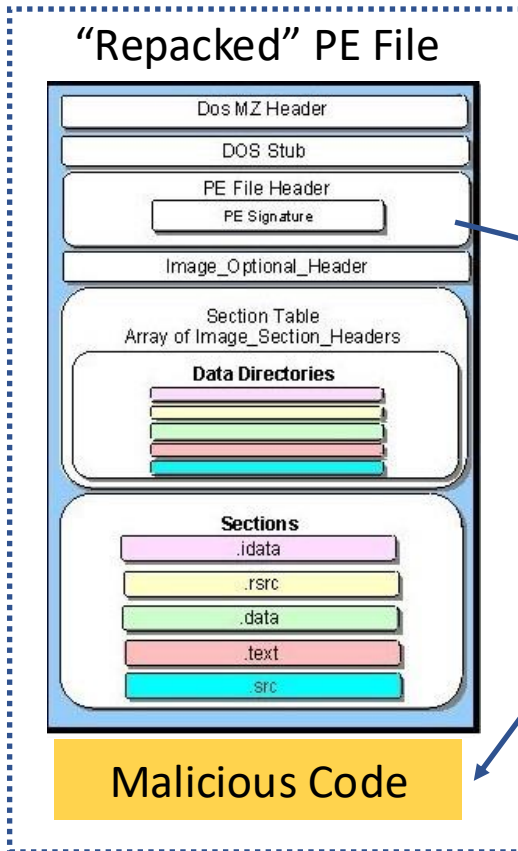


Viruses

- Is an attack that **modifies programs** on your host
- Approach (for Windows platform)
 - Download a program ...
 - Run the program ...
 - Searches for executables and other code (firmware, boot sector) that it can modify ...
 - Modifies these programs by **adding code that the program will run**

The Windows Executable File Format: Portable Executable (PE)

- How does virus work?
 - Modify the file executable format



1. Modify the header
2. Attach a “malicious” code section to PE file

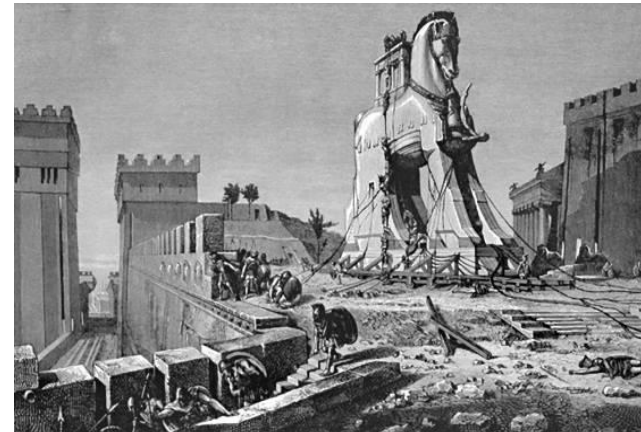
Redirect the execution
To malicious data section.

Where do Viruses Live?

- They live just about anywhere, such as...
- Boot sector
 - Take control before anything else
- Applications
- Library routines
- **Compilers**, debuggers, virus checker, etc.
 - These would be particularly nasty!

Trojan Horse

- **Trojan horse** is a malicious program that is disguised as legitimate software.
 - Like the gift horse left outside the gates of Troy by the Greeks, Trojan Horses appear to be **useful or interesting**, but are actually **harmful**.
 - But why the “horse” in the figure appears useful? I don’t know ;-)
 - Conceptually like **virus** with some subtle differences
 - Does not duplicate itself
 - Can usually be controlled remotely



Mac Trojan

- Prototype trojan for the Mac
- File icon for freeMusic.mp3:
 - Can use other accepted file extensions (e.g. .WSF, .WSH, .HTA, .PUB files)

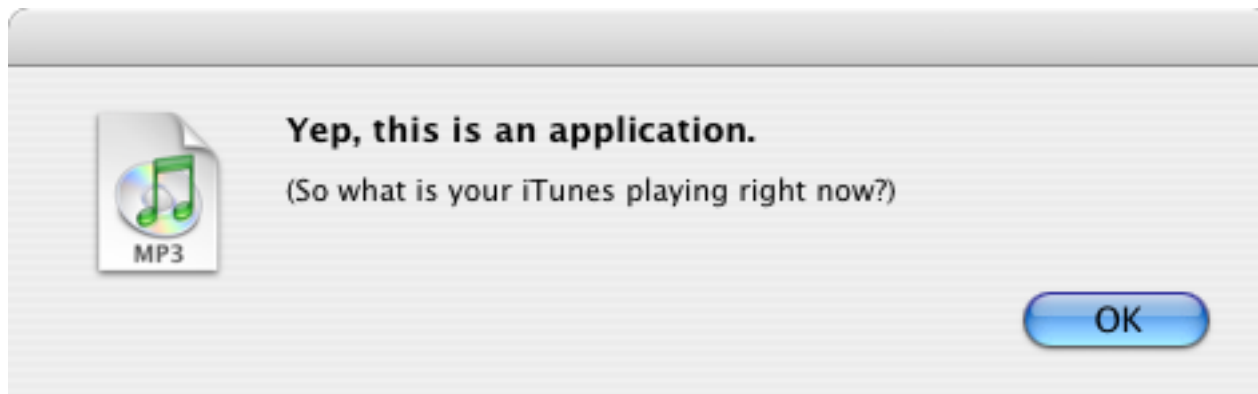


freeMusic.mp3

- For a real mp3, double click on icon
 - iTunes opens
 - Music in mp3 file plays
- But for freeMusic.mp3, unexpected results...

Mac Trojan

- Double click on freeMusic.mp3
 - iTunes opens (expected)
 - “Wild Laugh” (not expected)
 - Message box (not expected)



Mac Trojan

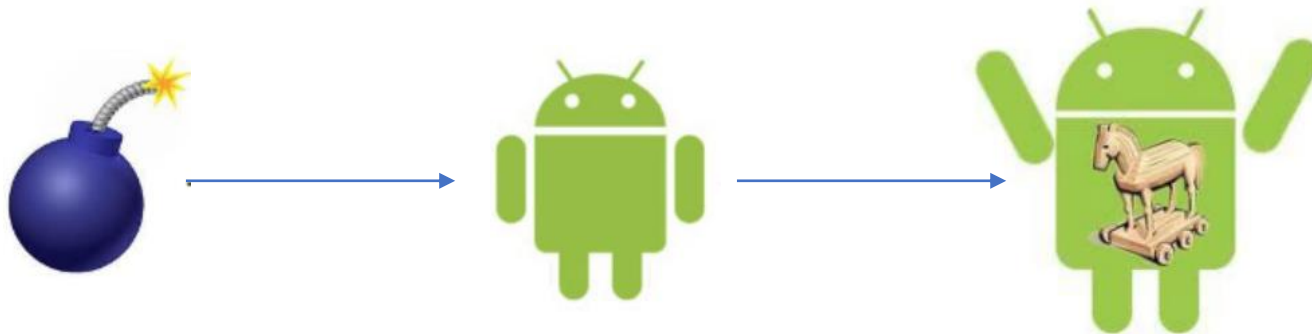
- How does freeMusic.mp3 trojan work?
- This “mp3” is an application, not data



- ❑ This trojan is harmless, but...
- ❑ ...could have done anything user could do
 - Delete files, download files, launch apps, etc.

Trojan in Android: App Repackaging Attack

- One of the most popular “modern” trojan horse...
 - Create a malicious **attack code fragment**
 - Download popular app and then decompile
 - Insert **attack code fragment** and distribute
- Don't download Android app from untrusted resource!!

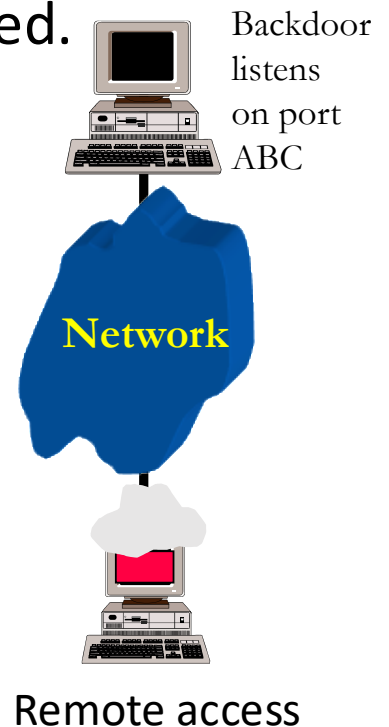


Backdoor

- Secretly bypass normal authentication in a computer system
 - Let malicious users to be remotely/locally access the computing system without being authenticated.

```
LDR      R0, =aScTUUuSSipSDip ; ">>> %s(ct=%u, un='%s',  
LDR      R1, =aAuth_admin_int ; "auth_admin_internal"  
BL       sub_558F74  
  
          ; CODE XREF: auth_admin_internal+2C↑j  
ADD      R0, R5, #0x44  
LDR      R1, =aSUnSU ; "<<< %s(un='%s') = %u"  
BL       strcmp  
CMP      R0, #0  
BNE      loc_13DC78  
MOV      R0, #0xFFFFFFFF  
LDMDDB   R11, {R4-R8,R11,SP,PC}
```

Local password authentication



“Reflections on Trusting Trust”

- The “root trust”
 - Ken Thompson on his 1984, Turing Award Lecture
 - Pioneer; (one) creator of UNIX and C language
 - *To what extent should one trust a statement that a program is free of Trojan horses? ← here “Trojan horses” and “backdoor” is interchangeable.*
- **Malicious compiler** yields software with backdoor.
 - Happens in Apple XcodeGhost in 2015.
 - And not just compilers...
- “developers looked for local copies of Xcode”



Crypto Ransomware



Malware Detection

- Common detection methods
 - **Static** signature detection
 - **Dynamic** behavior detection
 - *Malware clustering (not a detection methods, but can speed up the detection a lot)*
- We briefly discuss each of these
 - And consider advantages...
 - ...and disadvantages

Signature Detection

- A **signature** may be a string of bits in exe
 - Might also use wildcards, hash values, etc.
- For example, W32/Beast virus has signature
83EB 0274 EB0E 740A 81EB 0301 0000
 - That is, this string of bits appears in virus
- We can search for this signature in all files
- If string found, have we found W32/Beast?
 - Very likely, since software is not “random” sequences of bytes.

Ransomware Detection

- Lightweight symmetric key crypto is used to encrypt victim's data
 - TEA; twofish; blowfish; ...

TEA Encryption

Assuming 32 rounds:

$(K[0], K[1], K[2], K[3]) = 128 \text{ bit key}$

$(L, R) = \text{plaintext (64-bit block)}$

$\text{delta} = 0x9e3779b9$

Magic number!

sum = 0

for $i = 1$ to 32

sum += delta

$L \oplus= ((R \ll 4) + K[0]) \oplus (R + \text{sum}) \oplus ((R \gg 5) + K[1])$

$R \oplus= ((L \ll 4) + K[2]) \oplus (L + \text{sum}) \oplus ((L \gg 5) + K[3])$

ciphertext = (L, R)

000005c0:	0055	4889	e574	0c48	8b3d	3a0a	2000	e80d	.UH..t.H.=:. ...
000005d0:	ffff	ffe8	48ff	ffff	c605	310a	2000	015d	H.....1. ..]
000005e0:	c30f	1f80	0000	0000	f3c3	660f	1f44	0000	f..D..
000005f0:	5548	89e5	5de9	66ff	ffff	5548	89e5	c745	UH..].f...UH...E
00000600:	fc9b	7937	9e90	5dc3	0f1f	8400	0000	0000	..y7..].....
00000610:	4157	4156	4989	d741	5541	544c	8d25	ce07	AWAVI..AUATL.%..
00000620:	2000	5548	8d2d	ce07	2000	5341	89fd	4989	.UH.-.. .SA..I.
00000630:	f64c	29e5	4883	ec08	48c1	fd03	e877	feff	.L).H...H....w..
00000640:	ff48	85ed	7420	31db	0f1f	8400	0000	0000	.H..t 1.....

Pattern matching **delta**

Static Signature Detection

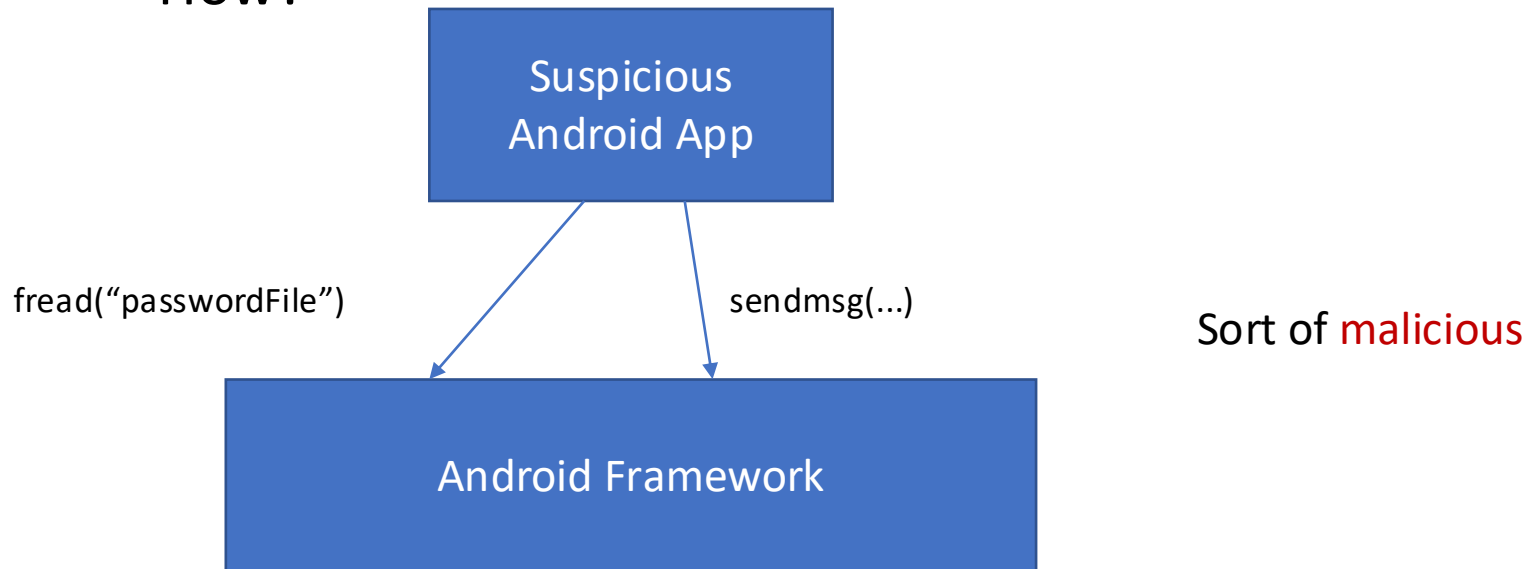
- Advantages
 - Effective on “ordinary” malware
 - Minimal burden for users/administrators
- Disadvantages
 - Signature file can be **large** (10s of thousands)...
 - ...making scanning slow
 - Signature files must be kept up to date
 - ***Cannot detect viruses if no specific signature is on hand***
 - Cannot detect some advanced types of malware
- The most popular detection method
 - And also the easiest

Dynamic Behavior Detection

- Monitor system for anything “unusual” or “virus-like” or “potentially malicious” or ...
- Examples of “unusual” things
 - Files access in some unexpected way
 - System misbehaves in some way
 - Unexpected network activity
- But we must first define “normal” behavior
 - And normal can (and must) change over time

Malware in Android

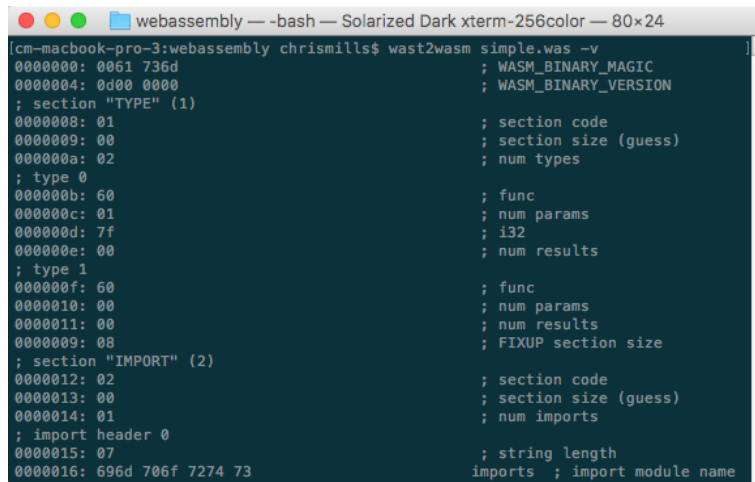
- Log the system call sequences as a way to reflect “malicious” behaviors.
 - How?



However, it's difficult to figure out the content being sent out with `sendmsg` => we will talk about that in security static analysis.

Malware in-browser mining

- Stealthily mining bitcoin in your browsers in the era of **webassembly**
 - But we pinpoint **a large amount of “mining” operations** during runtime (crypto hashing computation).



```
webassembly — -bash — Solarized Dark xterm-256color — 80x24
[cm-macbook-pro-3:webassembly chrismills$ wasm2wasm simple.wasm -v
00000000: 0061 736d                                ; WASM_BINARY_MAGIC
00000004: 0d00 0000                                ; WASM_BINARY_VERSION
; section "TYPE" (1)
00000008: 01                                         ; section code
00000009: 00                                         ; section size (guess)
0000000a: 02                                         ; num types
; type 0
0000000b: 60                                         ; func
0000000c: 01                                         ; num params
0000000d: 7f                                         ; i32
0000000e: 00                                         ; num results
; type 1
0000000f: 60                                         ; func
00000010: 00                                         ; num params
00000011: 00                                         ; num results
00000009: 08                                         ; FIXUP section size
; section "IMPORT" (2)
00000012: 02                                         ; section code
00000013: 00                                         ; section size (guess)
00000014: 01                                         ; num imports
; import header 0
00000015: 07                                         ; string length
00000016: 696d 706f 7274 73                         imports ; import module name
```

Webassembly

Practical Issues

- But, how do we actually get the “behavior” of a suspicious software?
 - Wait until damages happened?
 - Do it earlier but how to run a malicious software, observe its “malicious behavior” **without being damaged?**
- **Isolation the execution of suspicious software!**
 - Virtual machine
 - Sandbox
 - Hardware supports
- Then, can malware evade such detection?
 - Yes! Check if you are in a virtual environment
 - “sandbox evading”

The Matrix's Red Pill or Blue Pill



Dynamic Behavior Detection

- Advantages
 - Chance of detecting unknown malware with no static signatures (e.g., a hash) on hand
- Disadvantages
 - Accuracy?
 - Attacker can make “abnormal” behavior looks normal (being slow to attack)
 - Usually combine with another method (e.g., signature detection)

Malware Clustering

- People has accumulated a large amount of “known” malware samples
 - E.g., VirusTotal, the largest malware database
 - You can even get a membership of this database
- Then, given you a “unknown” software
 - Instead of knowing nothing and analyzing from scratch
 - We try to find whether it is similar to existing malware families.
 - Speedup when analyzing millions of unknown samples..

Malware Family

- People has accumulated a large amount of “known” malware samples
 - And many malwares are written by the same author or using the same malware toolkit.

TABLE III. 12 MOST FREQUENT MALWARE FAMILIES

The family name from Kaspersky	Counts
Backdoor.Win32.Hlux	27
HEUR:Trojan-Downloader.Win32.Generic	41
HEUR:Trojan.Win32.Generic	578
HEUR:Trojan.Win32.Invader	53
Packed.Win32.Tpyn	20
Trojan-Clicker.Win32.Agent	39
Trojan-Downloader.Win32.Agent	41
Trojan-Dropper.Win32.Injector	18
Trojan.Win32.Agent	196
Trojan.Win32.AntiFW	1480
Trojan.Win32.Buzus	46
Trojan.Win32.Inject	28

Malware Clustering

- Reuse software engineering/data mining techniques to identify the similarity of suspicious software
 - Similarity; clustering; embedding...

8b2e701e91101955c73865589a4c72999aeabc11043f712e05fdb1c17c4ab19a

62 / 71

62 engines detected this file

8b2e701e91101955c73865589a4c72999aeabc11043f712e05fdb1c17c4ab19a

ShellExtension

direct-cpu-clock-access peexe runtime-modules

249.00 KB
Size

2021-02-22 18:35:42 UTC
8 days ago

EXE

Community Score

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY 11
Acronis	Suspicious	Ad-Aware	Trojan.Foreign.Gen.2	
AhnLab-V3	Trojan/Win32.Agent.C578913	Alibaba	TrojanSpy:Win32/Obfuscator.c21d32b2	
ALYac	Trojan.Foreign.Gen.2	Antiy-AVL	Trojan[Spy]/Win32.Zbot	
SecureAge APEX	Malicious	Arcabit	Trojan.Foreign.Gen.2	
Avast	Win32:Cryptor	AVG	Win32:Cryptor	
Avira (no cloud)	TR/Spy.Zbot.ikzv	BitDefender	Trojan.Foreign.Gen.2	

Future of Malware

- Recent trends
 - Encrypted, polymorphic, metamorphic malware
 - Obfuscation/diversification (talk later)
- The future
 - It has become an “ecosystem” with steady revenue
 - Malware development as a “software engineering practice”..

The endless arms race: encrypted malware

- Malware writers know **signature detection** used
- So, how to evade signature detection?
- Encryption is a common approach
 - Ciphertext looks like random bits
 - Different key, then different “random” bits
 - So, different copies have no common signature
- Encryption often used in malware today

Encrypted Malware

- How to detect encrypted malware?
- Scan for the decryptor code
 - More-or-less standard signature detection
- Dump code content during runtime
 - Because the malware needs to decrypt itself before executing.

Metamorphic Malware

- A metamorphic malware mutates before infecting a new system
- Such a malware can, in principle, evade signature-based detection
- Mutated malware must function the same
 - And be “different enough” to avoid detection
- Detection is a bit more difficult
 - How ?

Metamorphic Malware

- One approach to metamorphic replication...
 - The malware **disassemble itself**
 - Random variations inserted into code (permute the code, insert dead code, etc.) ← we will talk about that.
 - Assemble the resulting code
- Result is a malware with same **functionality** as **original**, but different signature

AI in Malware

Principle, concept, and outlook

Malware clustering

- To against metamorphic malware
 - Clustering malware with the **same origin** in the same group.
 - Trivial signature (e.g., md5 hash) cannot work.



≈



Unknown
binary
(Metamorphic malware)

Known
ransomware

Simple rule-based

- An example
 - Yara rule malware detection
 - Summarizing detection rules for known malware

```
rule silent_banker : banker {  
  meta:  
    description = "Detects Silent Banker malware family"  
    author = "Malware Researcher"  
    date = "2025-07-21"  
    severity = 3  
  strings:  
    // Hex pattern unique to malware  
    $a = {6A 40 68 00 30 00 00 6A 14 8D 91}  
    // Obfuscated string  
    $b = "UVODFRYSIHLNWPEJXQZAKCBGMT" nocase  
    // Specific file path  
    $c = "C:\\Users\\admin\\AppData\\Local\\Temp\\banking.tmp" ascii  
  condition:  
    (uint16(0) == 0x5a4d) and ($a or $b or $c)}
```

A PE binary

Match at least one of the defined strings

Execution trace analysis

- **Concept:** measure the similarities of program **behavior** features.
- Typical steps [1]:
 - Dynamic analysis (use an emulator like Qemu)
 - Record the invocations, arguments, and return values of **system calls**
 - Behavior profile
 - Abstractions for system calls (call sequence and relevant OS objects)
 - Clustering
 - Binaries with similar behavior profiles are combined in the same cluster
 - The binaries in the malware cluster are likely malicious

ML-based analysis

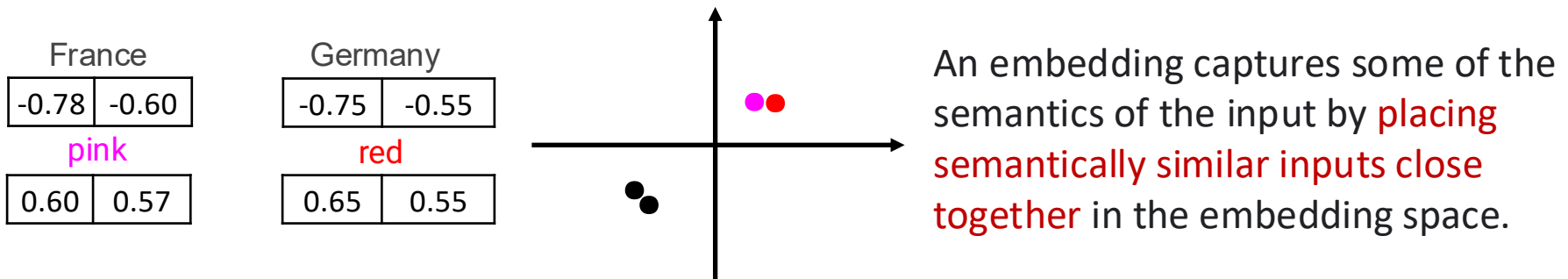
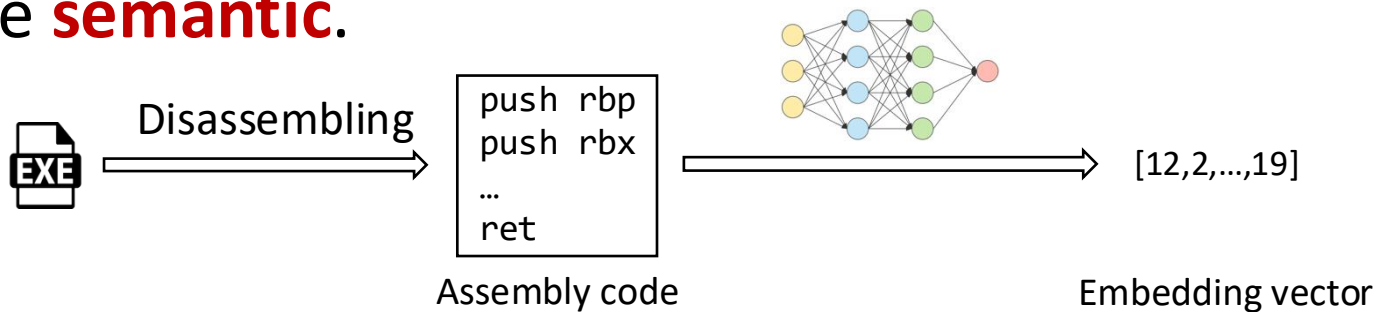
- Conventional study shows that **extracting program semantic features** for clustering malware.
- Machine learning is a good choice for extracting **semantic features/representations**.
 - E.g., word2vec [1], seq2seq [2]

[1] Efficient Estimation of Word Representations in Vector Space.

[2] Sequence to Sequence Learning with Neural Networks.

ML-based analysis (embedding)

- Learn an **embedding** for assembly code
- Binaries with the same **functionality** share the same **semantic**.



The embeddings of malware with the same origin will be close to each other.
Malware embeddings will be far from benign software embeddings.

ML-based

- Being inspired by natural language processing (NLP)

1	xorl	%eax, %eax	Good
2	testl	%edi, %edi	Job
3	jle .L1		!
4	movl	%edi, %eax	
5	cmpl	\$3, %edi	
6	jle .L1		
7	sall	\$2, %eax	
8	cmpl	\$8, %edi	
9	jle .L1		
10	addl	%edi, %eax	
11	.L1:		
12	ret		

Instruction = Word

Xu, Xiaojun, et al. "Neural network-based graph embedding for cross-platform binary code similarity detection." CCS 2017.

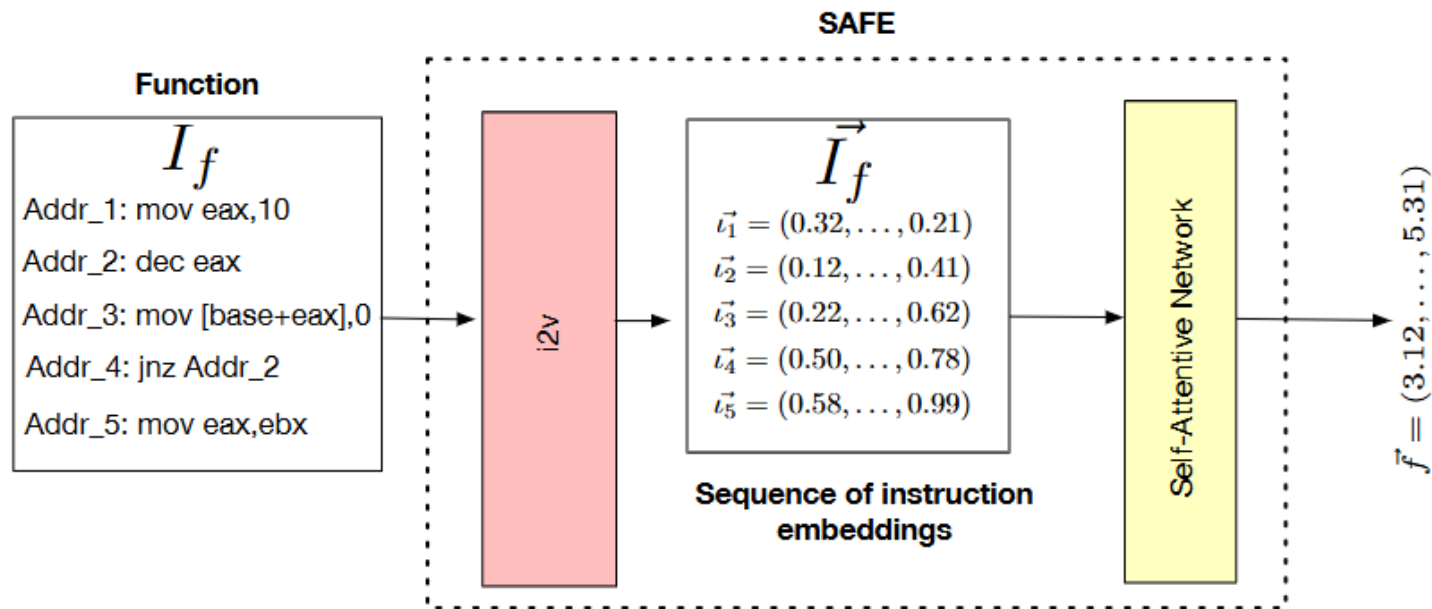
Luca Massarelli, et al. "SAFE: Self-Attentive Function Embeddings for Binary Similarity." DIMVA 2019.

Zuo, Fei, et al. "Neural machine translation inspired binary code similarity comparison beyond function pairs." NDSS 2018.

Wang, Hao, et al. "Jtrans: Jump-aware transformer for binary code similarity detection." ISSTA 2022.

ML-based

- Disassemble the binary code to get the assembly code
- Tokenize instructions
- Compute embeddings

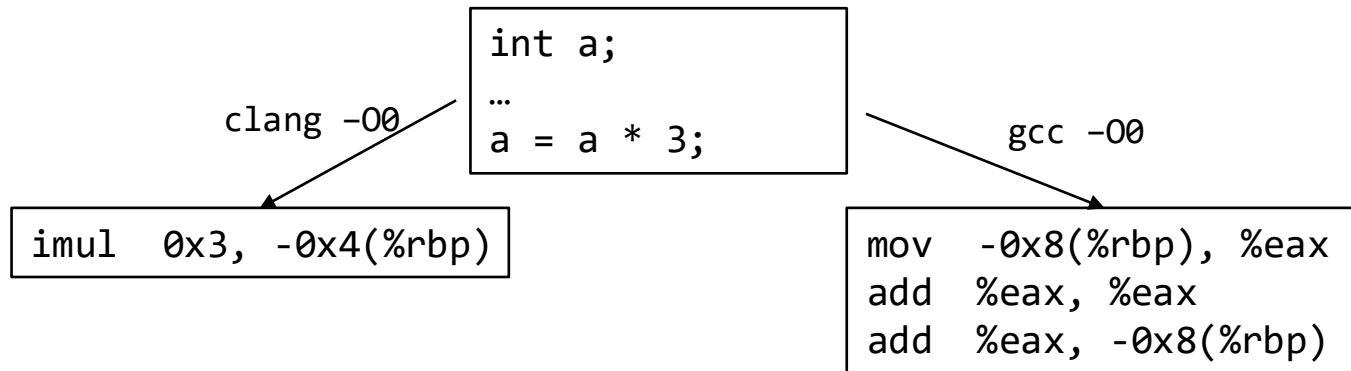


Training dataset construction

- The goal of the model
 - Produce similar embedding representations for different binary code with the same semantic (e.g., metamorphic malware)
- A large dataset that different binaries share the same functionalities.
 - Compiling the same source code with different configurations

Training dataset construction

- Various compilation optimizations produce different instructions



We can compile a project with different:

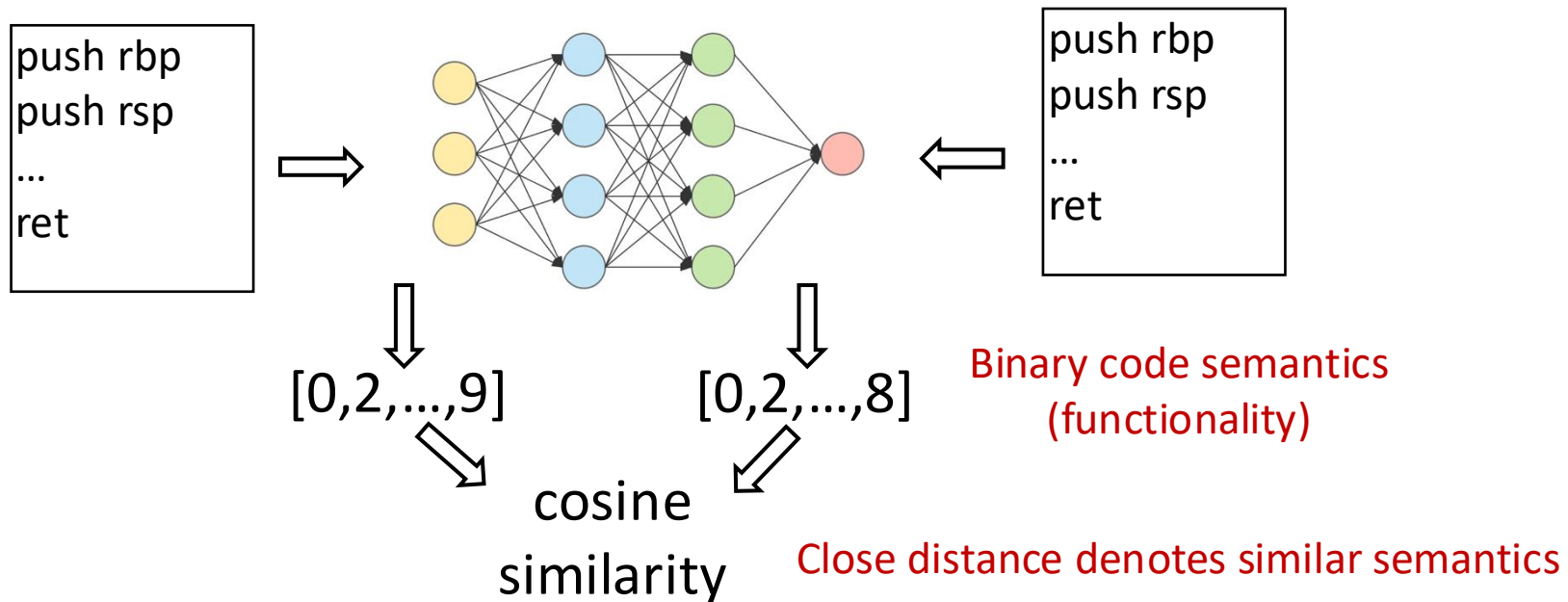
- compiler optimizations (-O0, -O3, -Os, etc)
- compilers (gcc, clang, icc, msvc, etc)
- for different architectures (Arm, x86, MIPS, etc)
- obfuscations

Training approach

- The goal of the model
 - Produce similar embeddings for different binary code with the same semantic (e.g., metamorphic malware)
- Training goal
 - Minimizing the embedding distance between binary code compiled from the same source code.
 - Maximizing the embedding distance between irrelevant binary code
 - Training data preparation:
 - Building binary code pairs and labels $(b1, b2, 1)$ or $(b1, b2, -1)$ for similar and dissimilar binary code pieces.

Model usage

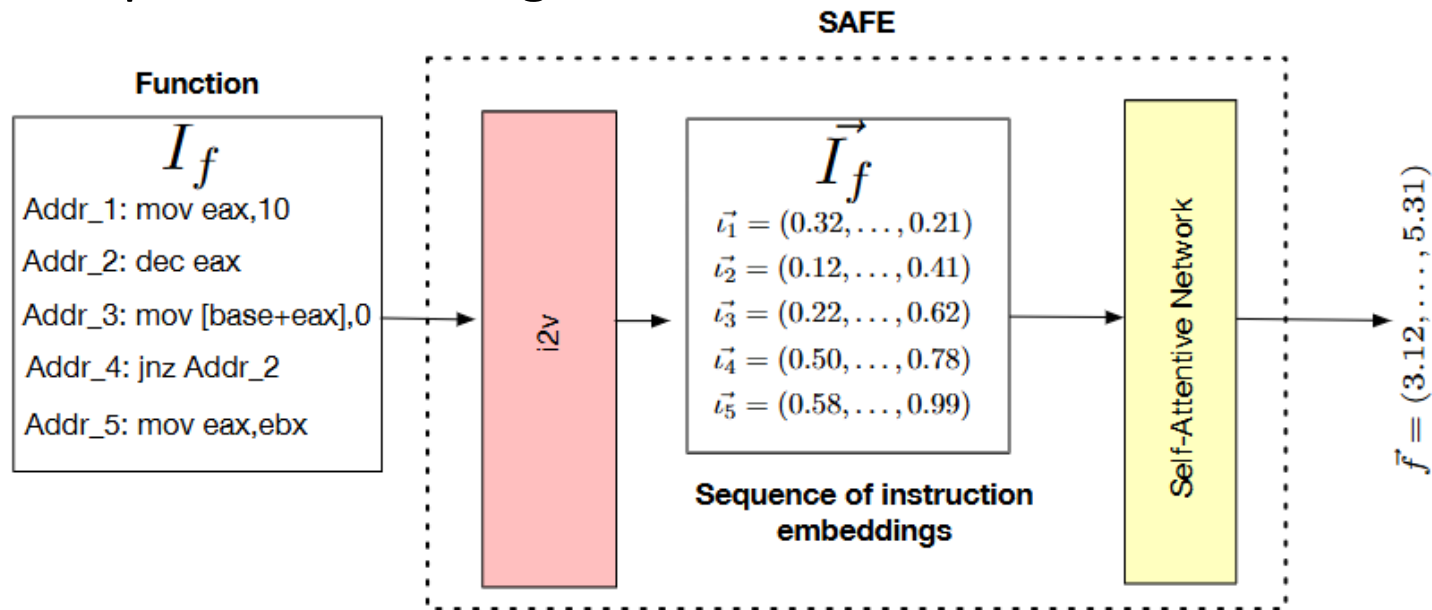
- Leveraging neural network (NN) to produce embeddings
- Embeddings with **high similarity** share similar functionalities



If a piece of binary code has a close embedding to the malicious code, it is likely malicious.

ML-based

- Disassemble the binary code to get the assembly code
- Tokenize instructions
- Compute embeddings

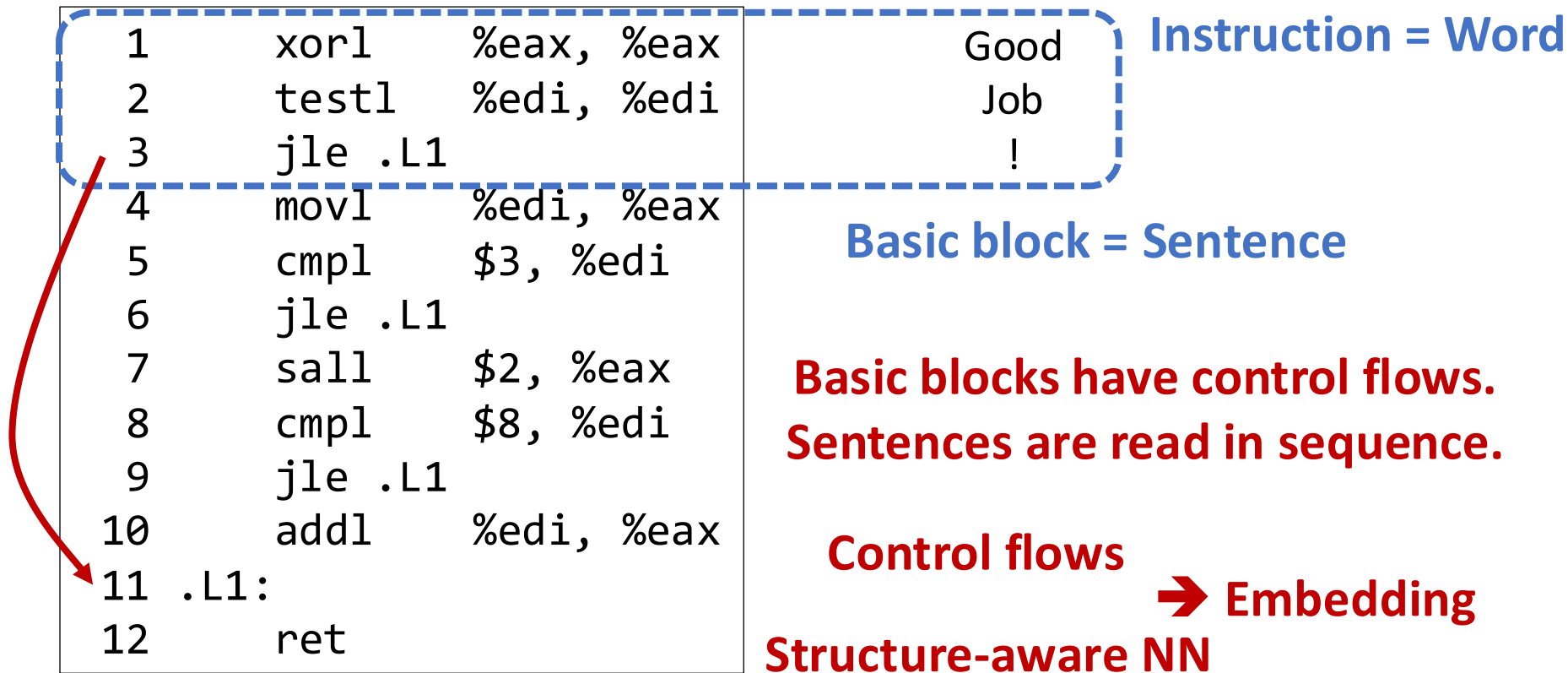


Efficient but less satisfactory accuracy

Ignoring the particular structure of code.

ML-based (Structure-aware)

- Being inspired by natural language processing

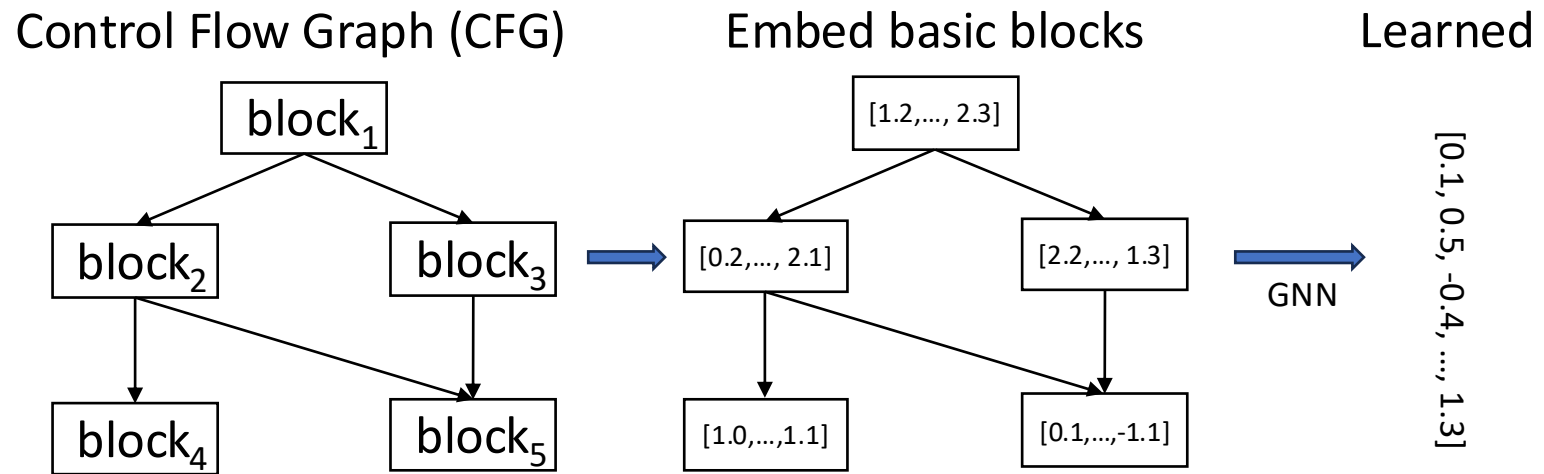


Xu, Xiaojun, et al. "Neural network-based graph embedding for cross-platform binary code similarity detection." CCS 2017.

Li, Xuezixiang, et al. "PalmTree: Learning an Assembly Language Model for Instruction Embedding." CCS 2021.

Wang, Hao, et al. "Jtrans: Jump-aware transformer for binary code similarity detection." ISSTA 2022.

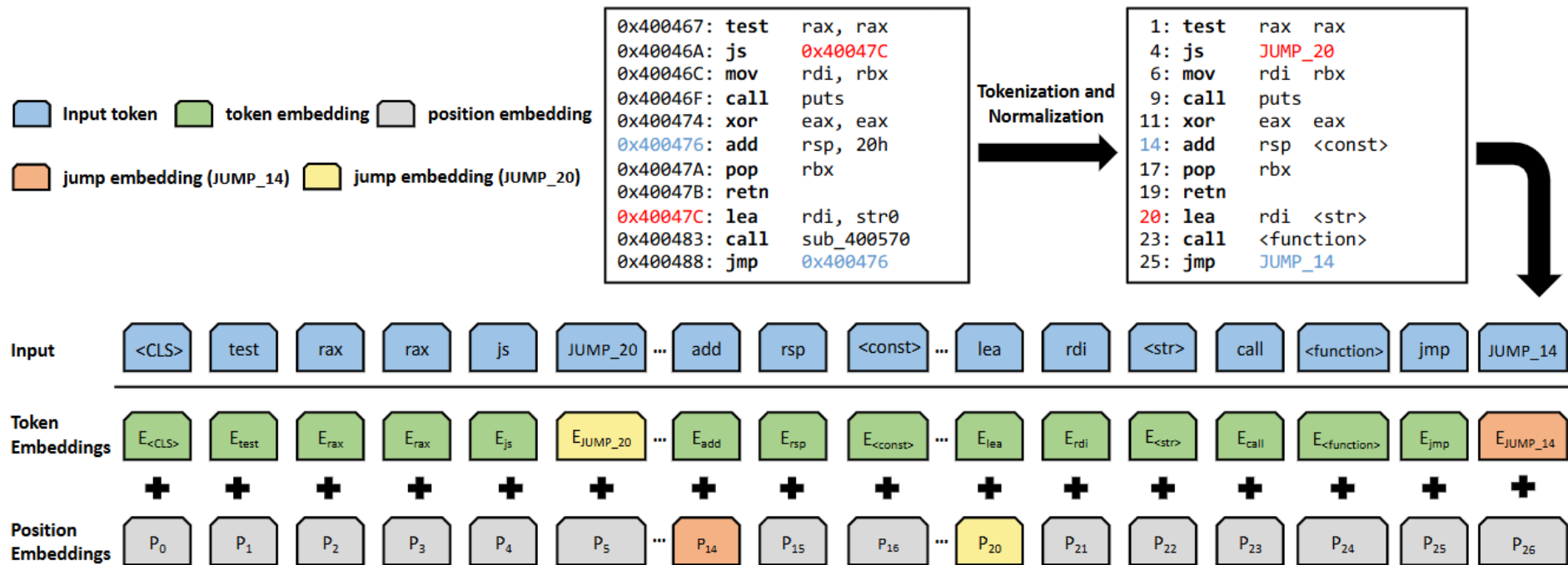
ML-based (GNN)



Embed a control flow graph with a graph neural network (GNN) [1, 2]

- [1] Xu, Xiaojun, et al. "Neural network-based graph embedding for cross-platform binary code similarity detection." CCS 2017.
- [2] Li, Xuezixiang, et al. "PalmTree: Learning an Assembly Language Model for Instruction Embedding." CCS 2021.

ML-based (Transformer)



LLM-enhanced

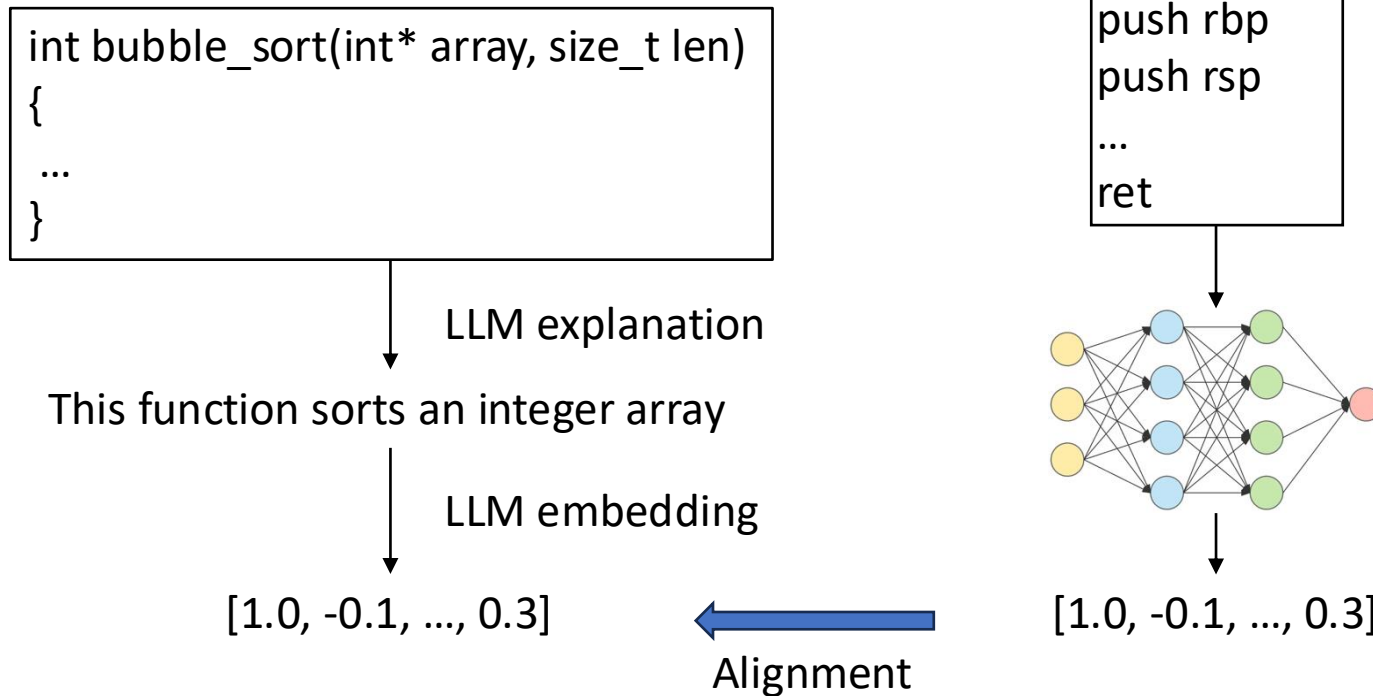
- The goal of the model
 - Produce similar embeddings for different binary code with the same semantic (e.g., metamorphic malware)
- Training goal
 - Minimizing the embedding distance between binary code compiled from the same source code.
 - Maximizing the embedding distance between irrelevant binary code
 - Training data preparation:
 - Building binary code pairs and labels (b1, b2, 1) or (b1, b2, -1) for similar and dissimilar binary code pieces.

E.g., Shall we maximizing the distance between bubble sort and quick sort?

Aligning binary code with natural language may address the issue and bring good performance.

LLM-enhanced

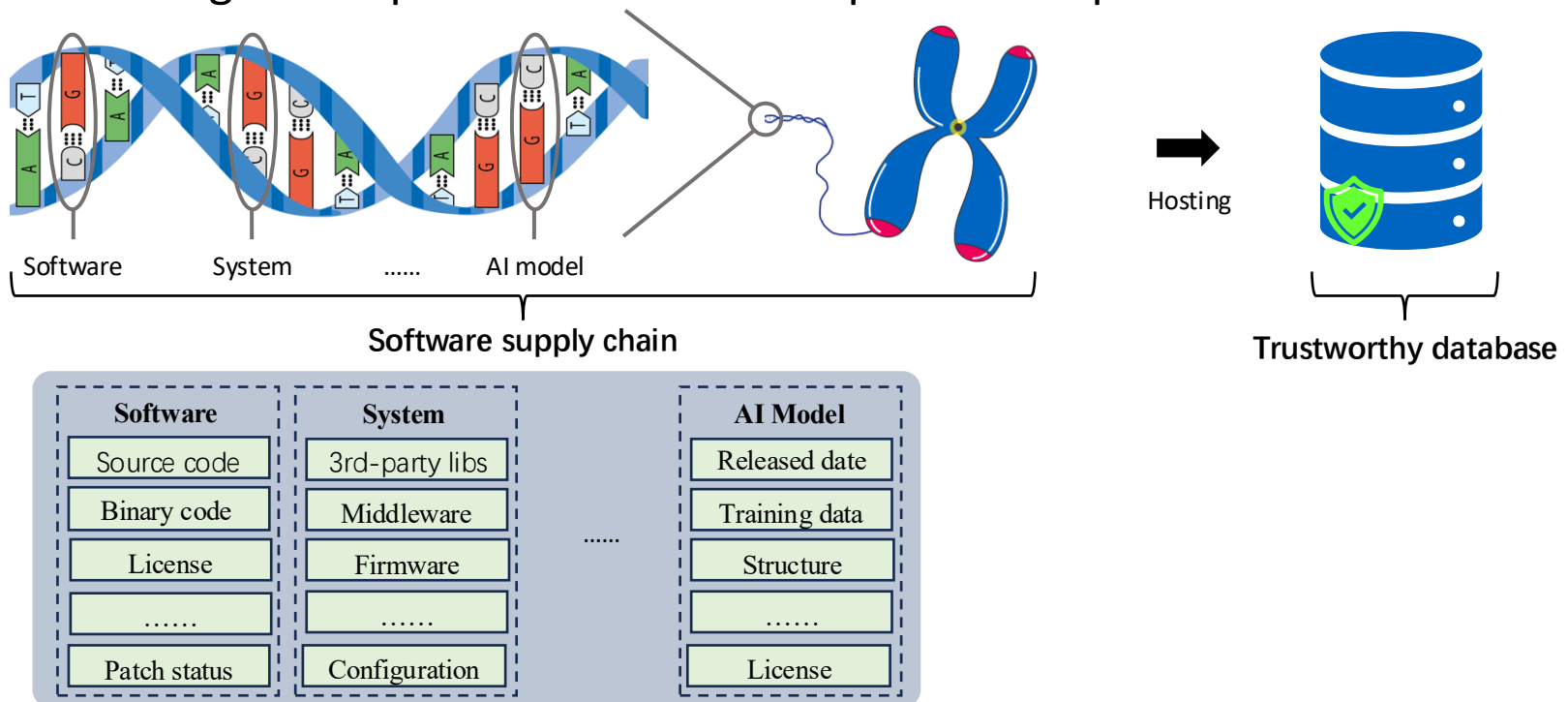
- Aligning assembly code embedding to the natural language explanation



Natural language embedding model was trained with extensive datasets.
Aligning the embedding of binary code with the natural explanation.

Software gene map

- (Malicious or benign) Software is evolving continuously
 - Becoming more complex and change rapidly
 - Techniques to search for their origins/ancestors are essential
 - A gene map for software can help us detect potential risks



ML-based method in practice

- Claude code-assisted, “vibe detection”.
 - Let’s focus on the core – **Binary Code Similarity Matching**
 - Of course I won’t download and manipulate a bunch of malware on my laptop....
- A glance of the state of the art [1]
 - This is rather complex, but I will try to walk through the key elements to shed light on SOTA.

[1] sem2vec: Semantics-Aware Assembly Tracelet Embedding. ACM TOSEM 2022